PROJECT PORTFOLIO / 21 OCTOBER 2026

Customer Data Quality Validation Pipeline

PySpark + PyDeequ + Docker

Mupembe Tshisekedi
Geology | Data Engineering | Data Science

I built a containerized customer-data validation workflow using native PySpark transformations and PyDeequ verification and profiling. The demonstrated run identifies repeated customer IDs, missing ages, and invalid age bounds, with saved JSON reports that reviewers can inspect and reproduce.

The problem

Missing values, repeated identifiers, and impossible customer attributes can distort analytical results. This project demonstrates how to detect and document those issues before adding automated publication controls. It uses deliberately imperfect synthetic data rather than production customer records.

Implemented workflow

Sample customer and reference records
Native PySpark checks and joins
PyDeequ VerificationSuite + AnalysisRunner
Verification and profiling reports in JSON

The workflow evaluates completeness, uniqueness, validity, reference accuracy, email consistency, and timeliness. Docker isolates Python 3.11, Java 17, Spark 3.5.3, and PyDeequ 1.4.0 from the host environment.

Reviewer links

Project repository, source code and README

Portfolio website | Successful automated run

Repository location: Rems2014/rems2014.github.io - projects/data-quality

Results that can be verified

Metric / check	Observed result	Interpretation
Sample size	4 records	Synthetic customer sample
Age completeness	0.75	Passes 75% threshold
Customer ID uniqueness	0.5	Fails unique-ID rule
Minimum age	-5	Fails age >= 0
Maximum age	150	Fails age <= 120
Mean non-null age	56.67	Null age excluded

Uniqueness 0.5 means two of four rows have customer IDs that occur exactly once. ID 2 occurs twice; this is one repeated ID and one excess record beyond the three distinct IDs. Failed data constraints are intentional findings, not execution failures.

Engineering decisions and trade-offs

Native Spark operations: Built-in expressions, filters, aggregations, and joins avoid Python row loops and Python UDFs. No benchmarked speedup is claimed.
Reproducibility: Docker and pinned packages avoid the host package conflicts encountered during setup.
Reviewable evidence: Small aggregate reports are collected to the driver and saved; the complete input is not collected for export.
Clean completion: Spark and the PyDeequ callback server are explicitly stopped after execution.

Verification and reproduction

The baseline local run produced saved reports. A fresh GitHub Actions run passed the Docker build, complete sample execution, both generated-report checks, and artifact upload. The portfolio website deployment also succeeded.

From the repository's **projects/data-quality** directory, with Docker Desktop running:

```
docker compose run --build --rm data-quality
python -m unittest discover -s tests -v
```

New reports: reports/verification.json and reports/metrics.json. Baseline evidence: examples/. The first run requires internet access to download dependencies.

Current limits and next steps

Spark runs locally with two worker threads inside one container. External API/database ingestion, Airflow, explicit ingestion schemas, Bronze/Silver persistence, and quarantine routing remain planned. Dataset-level failures do not automatically reject records or block publication. Reference joins do not fully cover missing keys or null postal codes; timeliness depends on the execution date. No production impact or large-scale performance benefit has been measured.